

# Principles of Programming Languages Compilation

Andrei Arusoaie<sup>1</sup>

<sup>1</sup>Department of Computer Science

# What are compilers?

- ▶ Programs that *translate* (usually) high-level code into low-level code

# What are compilers?

- ▶ Programs that *translate* (usually) high-level code into low-level code
- ▶ *source* code  $\rightarrow$  *target* code

# What are compilers?

- ▶ Programs that *translate* (usually) high-level code into low-level code
- ▶ *source* code → *target* code
- ▶ Compilers facilitate the work of the programmer: programmers focus only on the task to be solved rather than dealing with cumbersome low-level programming

# Compilers short history

- ▶ When symbolic languages and assemblers appeared the programs needed to be translated into machine code

# Compilers short history

- ▶ When symbolic languages and assemblers appeared the programs needed to be translated into machine code
- ▶ 1957, IBM: the first complete compiler of FORTRAN; portable on IBM computers.

# Compilers short history

- ▶ When symbolic languages and assemblers appeared the programs needed to be translated into machine code
- ▶ 1957, IBM: the first complete compiler of FORTRAN; portable on IBM computers.
- ▶ Later, COBOL was compiled on multiple architectures thanks to standard specifications and sustained efforts of various organisations

# Compilers short history

- ▶ When symbolic languages and assemblers appeared the programs needed to be translated into machine code
- ▶ 1957, IBM: the first complete compiler of FORTRAN; portable on IBM computers.
- ▶ Later, COBOL was compiled on multiple architectures thanks to standard specifications and sustained efforts of various organisations
- ▶ 1962, MIT: compiler for Lisp (although Lisp is known to be an interpreted language)
- ▶ 1970s: compilers for languages started to be implemented in the same language

# What compilers do?

- ▶ preprocessing, lexical analysis, parsing (+ disambiguation)

# What compilers do?

- ▶ preprocessing, lexical analysis, parsing (+ disambiguation)
- ▶ semantical analysis (type inference, type checking, object binding, initialisations for local variables)

# What compilers do?

- ▶ preprocessing, lexical analysis, parsing (+ disambiguation)
- ▶ semantical analysis (type inference, type checking, object binding, initialisations for local variables)
- ▶ control flow graph and corresponding analysis (dependencies, alias analysis, pointer analysis)

# What compilers do?

- ▶ preprocessing, lexical analysis, parsing (+ disambiguation)
- ▶ semantical analysis (type inference, type checking, object binding, initialisations for local variables)
- ▶ control flow graph and corresponding analysis (dependencies, alias analysis, pointer analysis)
- ▶ optimisations: inline expansions, macros, dead code elimination, constant propagation, loop transformation, automatic parallelisation

# What compilers do?

- ▶ preprocessing, lexical analysis, parsing (+ disambiguation)
- ▶ semantical analysis (type inference, type checking, object binding, initialisations for local variables)
- ▶ control flow graph and corresponding analysis (dependencies, alias analysis, pointer analysis)
- ▶ optimisations: inline expansions, macros, dead code elimination, constant propagation, loop transformation, automatic parallelisation
- ▶ code generation

# What compilers do?

- ▶ preprocessing, lexical analysis, parsing (+ disambiguation)
- ▶ semantical analysis (type inference, type checking, object binding, initialisations for local variables)
- ▶ control flow graph and corresponding analysis (dependencies, alias analysis, pointer analysis)
- ▶ optimisations: inline expansions, macros, dead code elimination, constant propagation, loop transformation, automatic parallelisation
- ▶ code generation
- ▶ others: error handling, generation of debug information, linking

# Compilers complexity

- ▶ Compilers are very complex and hard to implement

# Compilers complexity

- ▶ Compilers are very complex and hard to implement
- ▶ Challenge: make sure that compilers generate programs that are equivalent with the intended behaviour of the compiled programs

# Compilers complexity

- ▶ Compilers are very complex and hard to implement
- ▶ Challenge: make sure that compilers generate programs that are equivalent with the intended behaviour of the compiled programs
- ▶ Compilers can have bugs!
  - ▶ <https://web.cs.ucdavis.edu/~su/publications/issta16-compiler-bug-study.pdf>
  - ▶ Year 2016, found 39890 bugs in GCC, 12842 in LLVM
  - ▶ Most buggy code: C++ component

# Compilers bugs

- ▶ The expression  $(x == c1) \vee (x < c2)$  is equivalent to  $x < c2$  if  $c1$  and  $c2$  are constants and  $c1 \leq c2$

# Compilers bugs

- ▶ The expression  $(x == c1) \vee (x < c2)$  is equivalent to  $x < c2$  if  $c1$  and  $c2$  are constants and  $c1 \leq c2$
- ▶ LLVM bug:  $(x == 0) \vee (x < -3)$  was transformed into  $(x < -3)$  due to an unsigned comparison

**Source:** <https://www.cs.cornell.edu/courses/cs6120/2019fa/blog/bug-finding/>

# Compilers bugs

- ▶ **GCC:** <https://gcc.gnu.org/bugs/>

# Compilers bugs

- ▶ **GCC:** <https://gcc.gnu.org/bugs/>
- ▶ **Apple, the goto fail bug:**  
<https://www.synopsys.com/blogs/software-security/understanding-apple-goto-fail-vulnerability-2.html>

# Compilers bugs

- ▶ **GCC:** <https://gcc.gnu.org/bugs/>

- ▶ **Apple, the goto fail bug:**

[https:](https://www.synopsys.com/blogs/software-security/understanding-apple-goto-fail-vulnerability-2.html)

[//www.synopsys.com/blogs/software-security/understanding-apple-goto-fail-vulnerability-2.html](https://www.synopsys.com/blogs/software-security/understanding-apple-goto-fail-vulnerability-2.html)

- ▶ **Visual Studio 2007, floating point division bug:**

[https:](https://stackoverflow.com/questions/4282217/visual-c-weird-behavior-after-enabling-floating-rq=3)

[//stackoverflow.com/questions/4282217/visual-c-weird-behavior-after-enabling-floating-rq=3](https://stackoverflow.com/questions/4282217/visual-c-weird-behavior-after-enabling-floating-rq=3)

# Can theorem provers help?

- ▶ Interactive theorem provers: Coq, Isabelle

# Can theorem provers help?

- ▶ Interactive theorem provers: Coq, Isabelle
- ▶ Why?

# Can theorem provers help?

- ▶ Interactive theorem provers: Coq, Isabelle
- ▶ Why?
  - ▶ Mathematical proofs are often big and lacking details

# Can theorem provers help?

- ▶ Interactive theorem provers: Coq, Isabelle
- ▶ Why?
  - ▶ Mathematical proofs are often big and lacking details
  - ▶ Proofs can be developed using computers

# Can theorem provers help?

- ▶ Interactive theorem provers: Coq, Isabelle
- ▶ Why?
  - ▶ Mathematical proofs are often big and lacking details
  - ▶ Proofs can be developed using computers
  - ▶ Use a tool for handling all proof details

# Can theorem provers help?

- ▶ Interactive theorem provers: Coq, Isabelle
- ▶ Why?
  - ▶ Mathematical proofs are often big and lacking details
  - ▶ Proofs can be developed using computers
  - ▶ Use a tool for handling all proof details
  - ▶ Make sure proofs are **exact** - computer checked

# Can theorem provers help?

- ▶ Interactive theorem provers: Coq, Isabelle
- ▶ Why?
  - ▶ Mathematical proofs are often big and lacking details
  - ▶ Proofs can be developed using computers
  - ▶ Use a tool for handling all proof details
  - ▶ Make sure proofs are **exact** - computer checked
- ▶ These can be used to define compilers too and then prove properties about them!

# Outline

Introduction

Can theorem provers help?

Demo(s)

**Expressions**

Defining a stack machine

Compilation & Soundness

# Simple expressions in Coq

- ▶ Simple language of expressions - BNF:

$$\text{Exp} ::= \text{Nat} \mid \text{Id} \mid \text{Exp} \text{ "+" } \text{Exp} \mid \text{Exp} \text{ "*" } \text{Exp}$$

# Simple expressions in Coq

- ▶ Simple language of expressions - BNF:

$\text{Exp} ::= \text{Nat} \mid \text{Id} \mid \text{Exp} \text{ "+" } \text{Exp} \mid \text{Exp} \text{ "*" } \text{Exp}$

- ▶ (Abstract) syntax can be defined using `Inductive`:

```
Inductive Exp :=  
  | const : nat -> Exp  
  | id : string -> Exp  
  | plus : Exp -> Exp -> Exp  
  | times : Exp -> Exp -> Exp.
```

# Interpreter

- ▶ Expressions are interpreted in an environment, their interpretation being a value

# Interpreter

- ▶ Expressions are interpreted in an environment, their interpretation being a value
- ▶ The interpretation function:

```
Fixpoint interpret (e : Exp)
    (sigma : Var -> nat) : nat :=
match e with
| const x => x
| id s => sigma s
| plus e1 e2=>(interpret e1 sigma)+(interpret e2 sigma)
| times e1 e2=>(interpret e1 sigma)*(interpret e2 sigma)
end.
```

# Outline

Introduction

Can theorem provers help?

**Demo(s)**

Expressions

**Defining a stack machine**

Compilation & Soundness

# A stack machine

## ► Syntax:

```
Inductive Instruction :=  
| push_const : nat -> Instruction  
| push_var   : Var -> Instruction  
| add        : Instruction  
| mul        : Instruction.
```

# A stack machine

## ► Semantics:

```
Fixpoint run_instruction (i : Instruction)
  (env : Var -> nat) (stack : list nat) :=
  match i with
  | push_const n => n :: stack
  | push_var v  => (env v) :: stack
  | add         => match stack with
    | n1 :: n2 :: s' => (n1 + n2) :: s'
    | _              => stack
    end
  | mul         => match stack with
    | n1 :: n2 :: s' => (n1 * n2) :: s'
    | _              => stack
    end
  end.
```

# A stack machine

## ► Run instructions:

```
Fixpoint run (instr : list Instruction)
  (env : Var -> nat)
  (stack : list nat) : list nat :=
match instr with
| nil => stack
| i :: is' => run is' env
               (run_instruction i env stack)
end.
```

# Outline

Introduction

Can theorem provers help?

Demo(s)

Expressions

Defining a stack machine

**Compilation & Soundness**

# Compile

## ► Compilation of `Exp` to the stack machine:

```
Fixpoint compile (e:Exp) : list Instruction :=  
  match e with  
  | const n => push_const n :: nil  
  | id v => push_var v :: nil  
  | plus e1 e2 => (compile e2) ++ (compile e1)  
                  ++ (add :: nil)  
  | times e1 e2 => (compile e2) ++ (compile e1)  
                  ++ (mul :: nil)  
end.
```

# Compilation results

► Compilation correct:

**Theorem** `compilation_correct`:  
  **forall** e env,  
    run (compile e) env nil =  
      (interpret e env) :: nil.

# Compilation results

## ► Compilation correct:

**Theorem** `compilation_correct`:

```
forall e env,  
  run (compile e) env nil =  
    (interpret e env) :: nil.
```

## ► Invariant:

**Lemma** `compilation_correct'`:

```
forall e env is' stack,  
  run (compile e ++ is') env stack =  
    run is' env (interpret e env :: stack).
```

# Bottom line

- ▶ We have defined two languages in Coq: a language of expressions  $\text{Exp}$  and a stack machine
- ▶ We show how  $\text{Exp}$  can be compiled to the stack machine
- ▶ We **prove** compilation correct!

# CompCert - I

- ▶ X. Leroy, S. Blazy, Z. Dargaye, J.B. Tristan; et al.
- ▶ Link: <http://compcert.inria.fr/>
- ▶ Project: develop and prove a realistic compiler, usable for critical embedded software
- ▶ Source language: a very large subset of C
- ▶ Target language: PowerPC/ARM/x86 assembly
- ▶ Generates compact and fast code; careful code generation + some optimizations
- ▶ Compiler written from scratch + **proof**

# CompCert - II

- ▶ 50000 lines of Coq
- ▶ 4 person-years
- ▶ Trusted execution, certified compilation, static analysis (Verasco)
- ▶ Direct and indirect project partners: Airbus, AbsInt, etc.
- ▶ Airbus: flight control software of the A380
- ▶ MTU: critical control system in diesel engines that are deployed in nuclear power plants
- ▶ Institute of Flight System Dynamics at TUM: development of flight control and navigation algorithms

# Resources

1. Chapter Small-Step Operational Semantics in **Software Foundations - Volume 2**, Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, Brent Yorgey

Section “*A Small-Step Stack Machine*”:

[https://softwarefoundations.cis.upenn.edu/  
plf-current/Smallstep.html#lab169](https://softwarefoundations.cis.upenn.edu/plf-current/Smallstep.html#lab169)